Centro Universitário de Excelência
Sistemas de Informação

Explorando estruturas de dados não lineares

Autores: Lucas Matheus dos Santos Marques
Matheus Barbosa dos Santos Carvalho
Vitor Hugo da Silva Oliveira

Feira de Santana, 2026

Agenda

O objetivo dessa apresentação é demonstrar como compreendemos e implementamos estruturas não lineares baseadas em árvores para melhorar a performance do sistema da MecâniQA - Automotive Tech.

1. Desafio da performance :
Contextualizando e justificando as mudanças.

2. Estrutura da árvore:
Explicando sobre a composição dos nós da árvore binária.

3. Importância do equilíbrio:
Enfatizando a importância do balanceamento nesse tipo de estrutura.

4.Inserção e remoção :
Explicação dos algoritmos de balanceamento na árvore AVL.

5. Exibição e relatórios:
Uso dos percursos em ordem para listagem.

Desafio da performance

Problemas de performance relacionadas às buscas lineares

Uso de árvores binárias com nova regra de ordenação

Garantir o balanceamento para não virar O(n)

Buscas otimizadas com complexidade de O(log n)

Explorando estruturas de dados não lineares

Estrutura da árvore

- Itens para discussão:
  - Composição dos nós;
  - Regra de ordenação (esquerda e direita);
  - Flexibilidade da árvore (Generics<T>);
  - Código como critério de ordenação;
  - Busca otimizada O(log n).

```
public class NoArvore <T extends Entidade<T>> {
    public T registro;  12 usages
    public int id;  16 usages
    public int altura = 1;  7 usages
    public NoArvore<T> esquerda;  43 usages
    public NoArvore<T> direita;  40 usages
```

Estrutura da árvore

Comparativo

Busca linear

```java
public static int buscarPeca(int codigo){  2
    //varre o vetor procurando
    for (int i = 0; i < proximoIndex; i++){
        if (pecas[i].codigo == codigo)
            return i;
    }
    return -1;
}
```

Complexidade O(n)

Busca na BST

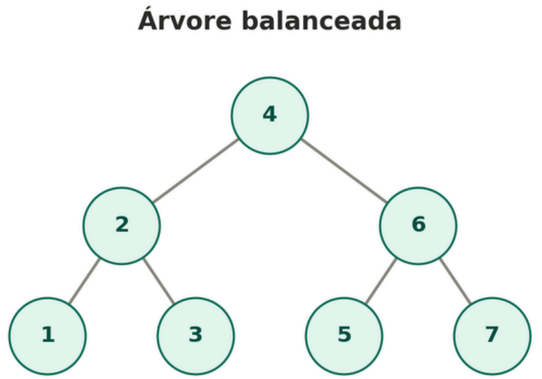
```java
private NoArvore<T> find(int codigo, NoArvore<T> atual) {
    if (atual == null) { //[cite: 2]
        return null;
    }
    if (atual.id == codigo) {
        return atual;
    } else if (atual.id > codigo) {
        return find(codigo, atual.esquerda);
    } else {
        return find(codigo, atual.direita);
    }
}
```

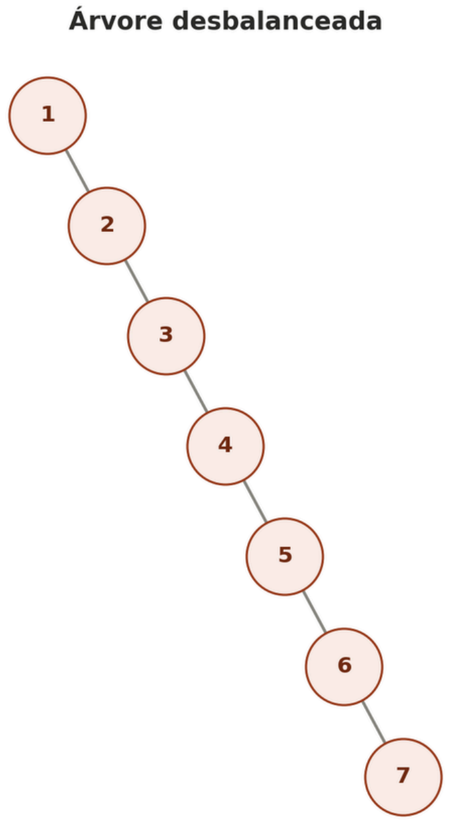
Complexidade O(log n) quando balanceada

Explorando estruturas de dados não lineares

Importância do equilíbrio

- Itens para discussão:
  - Desempenho prejudicado;
  - Cálculo da altura do nó;
  - Fator de balanceamento (-1,0,1);
  - Como garantir o equilíbrio na inserção e remoção?





Inserção e remoção

O que torna a árvore AVL eficiente?

1. inserir(NoArvore<T> atual, T registro)
2. Desce a árvore comparando os ids e inserindo no nó nulo
3. Volta da recursão e atualiza a altura do nó pai
4. Verifica o fator de balanceamento

```
// Casos de Rotação
// Esquerda Esquerda
if (balanceamento > 1 && registro.getCodigo() < atual.esquerda.id)
    return rotacaoDireita(atual);

// Direita Direita
if (balanceamento < -1 && registro.getCodigo() > atual.direita.id)
    return rotacaoEsquerda(atual);

// Esquerda Direita
if (balanceamento > 1 && registro.getCodigo() > atual.esquerda.id) {
    atual.esquerda = rotacaoEsquerda(atual.esquerda);
    return rotacaoDireita(atual);
}

// Direita Esquerda
if (balanceamento < -1 && registro.getCodigo() < atual.direita.id) {
    atual.direita = rotacaoDireita(atual.direita);
    return rotacaoEsquerda(atual);
}

return atual;
```

Inserção e remoção

Remoção é um pouco mais complexa

1.remover(NoArvore<T> atual, int codigo)

2.Ao achar o id, verifica se o nó é folha ou tem filhos

3. Ao excluir, atualiza a altura do nó pai e seu fator de balanceamento

4.Entra na mesma cadeia de verificações

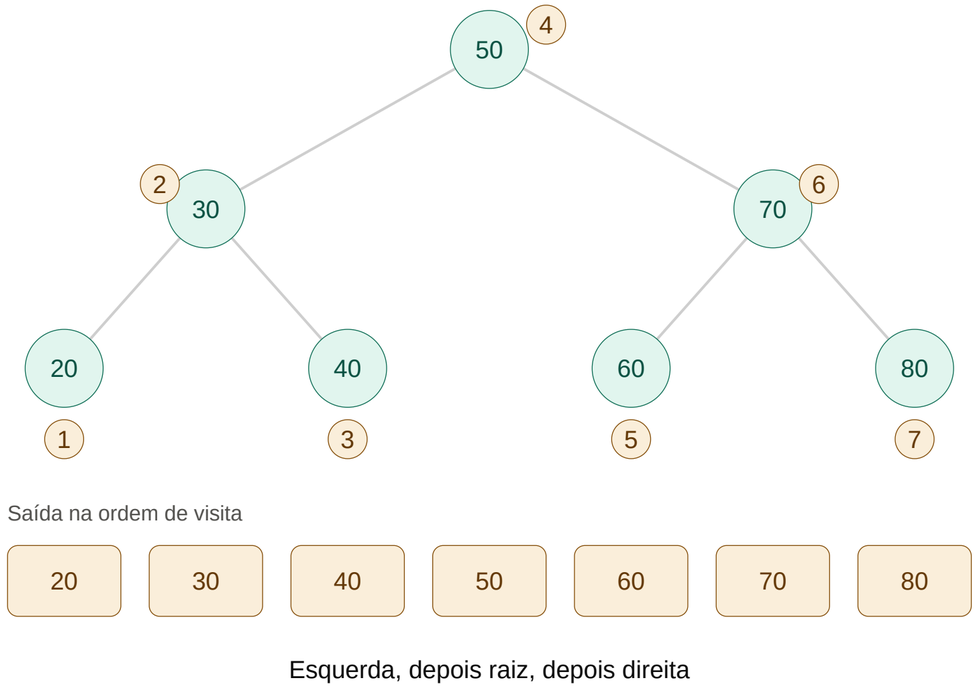
Caso 1:
Nenhum filho → retorna nulo na função

Caso 2:
Um filho →
1. Verifica qual lado não é nulo
2. Atribui o valor do nó atual ao nó filho não nulo
3. Retorna o nó atual com o novo valor na função

Caso 3:
Dois filhos →
1. Procura o maior valor na subarvore da esquerda
2. Atribui o valor do nó atual ao do maior valor encontrado
3. Chama a função de remoção para o nó de maior valor encontrado
4. Cai nos casos 1 ou 2

Exibição e relatórios

- Itens para discussão:
  - Qual algoritmo de travessia devemos usar?
  - Classe GerenciadorArvore executa a travessia;
  - Adição do objeto no CSV na criação;
  - Inserção do objeto na árvore na criação.
  - Remoção física no CSV ao remover da árvore e fazer a exportação.



Demonstração

```
Cliente c1  = new Cliente( nome: "Ana Souza",  telefone: "(75)
Cliente c2  = new Cliente( nome: "Bruno Lima",  telefone: "(75)
Cliente c3  = new Cliente( nome: "Carla Mendes",  telefone: "(7
Cliente c4  = new Cliente( nome: "Diego Santos",  telefone: "(7
Cliente c5  = new Cliente( nome: "Elisa Rocha",  telefone: "(75
Cliente c6  = new Cliente( nome: "Fábio Almeida",  telefone: "(
Cliente c7  = new Cliente( nome: "Gabriela Costa",  telefone: '
Cliente c8  = new Cliente( nome: "Henrique Alves",  telefone: '
Cliente c9  = new Cliente( nome: "Isabela Ferreira",  telefone
Cliente c10 = new Cliente( nome: "João Pereira",  telefone: "(7
```

Criação dos objetos

```
----------------------------------------------------------------
30;jo◆o;721513316;aksasdasdaajabs@gmail.com
31;Ana Souza;(75) 99111-0001;ana.souza@email.com
32;Bruno Lima;(75) 99111-0002;bruno.lima@email.com
33;Carla Mendes;(75) 99111-0003;carla.mendes@email.com
34;Diego Santos;(75) 99111-0004;diego.santos@email.com
35;Elisa Rocha;(75) 99111-0005;elisa.rocha@email.com
36;F◆bio Almeida;(75) 99111-0006;fabio.almeida@email.com
37;Gabriela Costa;(75) 99111-0007;gabriela.costa@email.com
38;Henrique Alves;(75) 99111-0008;henrique.alves@email.com
39;Isabela Ferreira;(75) 99111-0009;isabela.ferreira@email.com
40;Jo◆o Pereira;(75) 99111-0010;joao.pereira@email.com
41;Ana Souza;(75) 99111-0001;ana.souza@email.com
42;Bruno Lima;(75) 99111-0002;bruno.lima@email.com
43;Carla Mendes;(75) 99111-0003;carla.mendes@email.com
44;Diego Santos;(75) 99111-0004;diego.santos@email.com
```

Lista de clientes imprimidos no console em ordem crescente

```
Cliente.arvoreCliente.exibirOrdem();
```

Chamada do método exibirOrdem()

Explorando estruturas de dados não lineares

Considerações finais

- Nessa etapa da OAT acompanhamos o ritmo de crescimento da MecaniQA;
- Implementamos estruturas capazes de fornecer de forma quase imediata o acesso aos dados;
- Tudo isso graças ao uso de árvores binárias de busca com sua complexidade de tempo de O(log n) nas operações de busca, reduzindo o gargalo do sistema.

Referências

- Caderno de guias da OAT 3: UNEX- Estruturas de dados
- https://www.youtube.com/watch?v=3zmjQlJhBLM
- Repositório do projeto: https://github.com/vitorhugosilvaoliveira/mecaniQA-palmas
- https://www.youtube.com/watch?v=A-7x7CPuolM&t=217s
- Aulas do Prof. Lucas Almeida Silva disponíveis no blackboard da FTC.